

DATABASE MANAGEMENT SYSTEMS • MAJOR PROJECT

Warehouse Network Optimization System

A relational data model and analytics layer that decides how many warehouses a network should have, where they belong, what each one stocks, and which lane every shipment moves on.

26

Entities

39

Relationships

25

Analytical queries

11.28%

Best cost saving

AUTHOR

Aayushman Ghatak

The Problem

Three cost blocks that move in opposite directions

1 Facility cost

Fixed rent, staff and handling per warehouse.

▼ Falls when you operate fewer, larger nodes.

▲ Rises with every node you open.

2 Transportation cost

Freight, tolls, fuel and congestion on every lane.

▼ Falls when nodes sit close to demand.

▲ Rises when you centralise.

3 Inventory carrying cost

Capital locked in stock, at roughly 22% a year.

▼ Falls when you centralise (risk pooling).

▲ Rises when stock is fragmented.

Add a warehouse and freight falls but fixed cost rises and safety stock fragments. Close one and the reverse happens. No single department can see the whole trade-off — which is exactly what a shared database is for.

Existing practice: warehouse siting decided once, by intuition, then left untouched for years — while demand, fuel prices and customer geography all move.

Objectives

What the proposed system must be able to do

1

Model the physical network

Facilities, storage zones, SKUs, suppliers, customers, lanes, carriers and fleet in one normalised schema (3NF / BCNF).

2

Hold a live, auditable stock position

One row per warehouse-SKU, plus an immutable signed transaction ledger so any historical position can be replayed.

3

Forecast demand and measure the forecast

Store the projection and the realised actual, so MAPE is a queryable column and not a claim.

4

Choose lanes and modes on cost

Road, rail, air and multimodal arcs for the same city pair, priced with tolls and a congestion index.

5

Make optimizer output first-class data

Scenarios, runs, algorithms and per-node recommendations are stored — so results are reproducible and comparable.

6

Expose total network cost as one query

The objective function evaluated directly from live tables (Q20), not from a spreadsheet.

Existing System vs Proposed System

Why a spreadsheet stops working at network scale

Dimension	Existing practice	Proposed system (WNOS)
Data storage	Spreadsheets per warehouse, emailed monthly	Single normalised MySQL schema, 26 related entities
Stock accuracy	Manually keyed; no audit trail	Generated columns + immutable transaction ledger
Order allocation	Nearest warehouse, by habit	Recorded method: NEAREST / LOWEST_COST / LOAD_BALANCED / OPTIMIZER
Transport choice	One incumbent road carrier	Multi-modal lanes ranked on effective cost and transit time
Demand planning	Last year plus a percentage	Six forecast models with stored MAPE per warehouse-SKU
Network design	Reviewed every few years, if at all	Scenario + run + recommendation tables; re-runnable any day
Integrity	Enforced by convention	39 FK constraints, 32 CHECK constraints, 13 indexes

System Architecture and Technology Stack

Four layers, with the database as the system of record



Data Layer at a glance

26	tables
39	relationships
32	CHECK constraints
13	secondary indexes
3	reporting views
25	analytical queries

Conceptual ER Model

26 entities organised into 7 functional modules

Geography & Network

region · location · warehouse · warehouse_zone

Product & Supply

product_category · product · supplier · supplier_product · customer

Inventory

inventory · inventory_transaction

Demand & Orders

customer_order · order_line · order_fulfilment · demand_forecast

Transportation

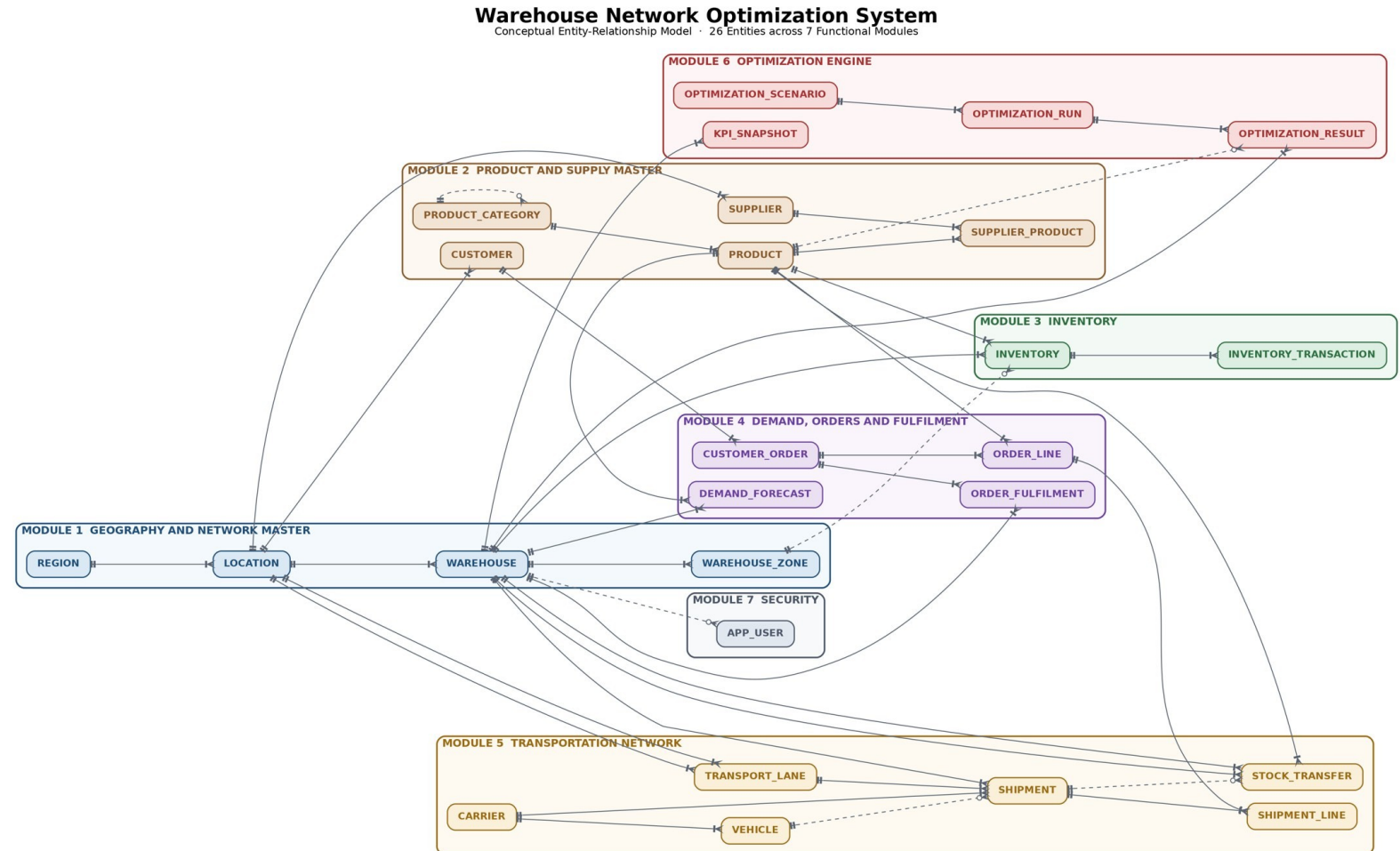
carrier · vehicle · transport_lane · shipment · shipment_line · stock_transfer

Optimization

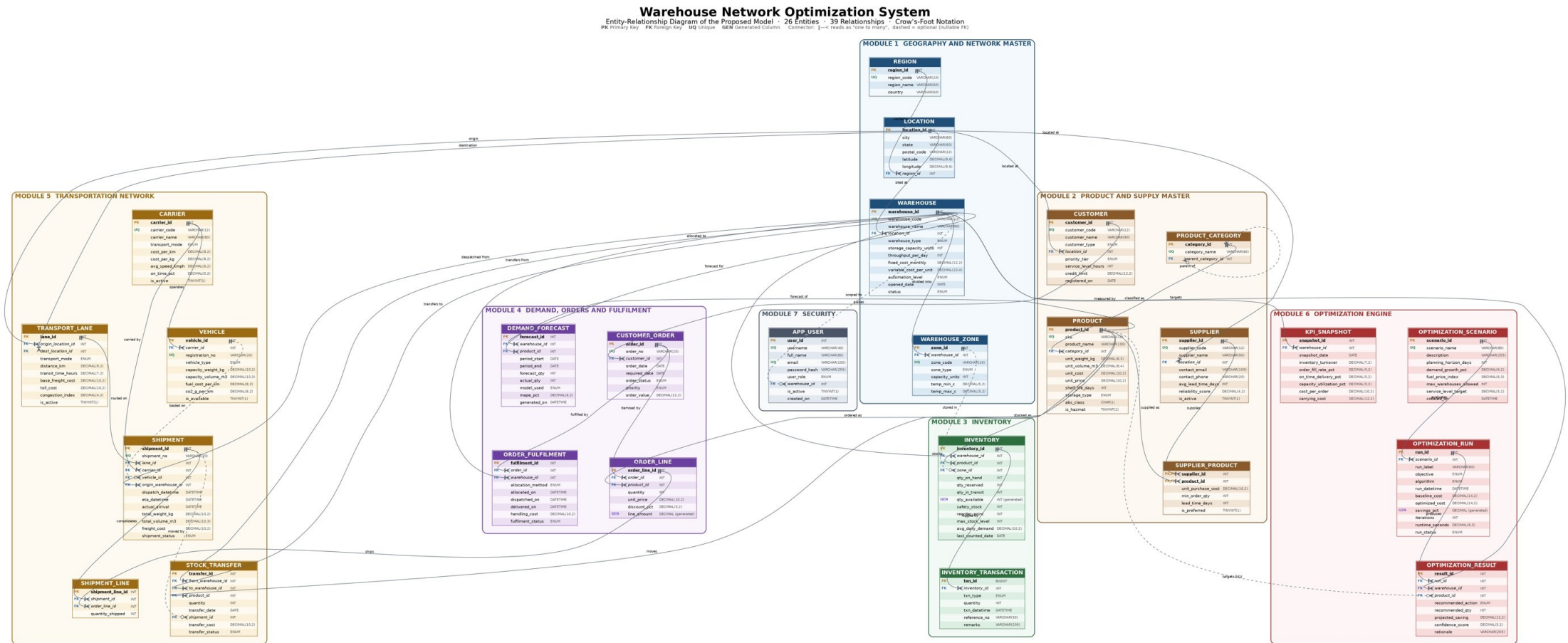
optimization_scenario · optimization_run · optimization_result · kpi_snapshot

Security

app_user



Attribute level · crow's-foot notation · PK, FK, UQ and generated columns marked



Solid connector = mandatory foreign key. Dashed connector = optional (nullable) foreign key. Full-resolution PNG, PDF and SVG are supplied in the diagrams folder.

Schema Design and Normalisation

Every table in BCNF, with two deliberate exceptions

1NF

Every attribute atomic. Multiple zones, order lines or lanes become rows in child tables — never comma-separated columns.

2NF

In `supplier_product` (PK = `supplier_id` + `product_id`), `unit_purchase_cost` depends on both parts; `supplier_name` depends on `supplier_id` alone, so it stays in `supplier`.

3NF

`warehouse` stores `location_id`, not `city` / `state` / `region_name` — those depend on `location_id`, not on `warehouse_id`.

BCNF

Every determinant is a candidate key: `warehouse_code`, `sku`, `order_no` and `shipment_no` are all declared UNIQUE.

Controlled redundancy — on purpose

`order_line.unit_price`

Duplicates `product.unit_price` at the moment of sale. Not a 3NF violation — it is a temporal attribute. Repricing a SKU must not rewrite history.

Generated columns

`qty_available`, `line_amount` and `savings_pct` are derived, but computed by the engine, so they cannot drift from their inputs.

`customer_order.order_value`

A materialised aggregate maintained by an explicit UPDATE ... JOIN — read speed traded for one extra write.

Referential integrity policy: RESTRICT on master data that transactions point at · CASCADE on true parent-child ownership (`warehouse`→`zone`, `order`→`line`, `run`→`result`) · SET NULL on optional associations (`inventory`→`zone`, `shipment`→`vehicle`).

DDL — Data Definition Language

01_schema_ddl.sql · 26 CREATE TABLE statements, 3 views

```
CREATE TABLE inventory (  
  inventory_id      INT      NOT NULL AUTO_INCREMENT,  
  warehouse_id     INT      NOT NULL,  
  product_id       INT      NOT NULL,  
  zone_id          INT      NULL,  
  qty_on_hand      INT      NOT NULL DEFAULT 0,  
  qty_reserved     INT      NOT NULL DEFAULT 0,  
  qty_available    INT AS (qty_on_hand - qty_reserved) STORED,  
  safety_stock     INT      NOT NULL DEFAULT 0,  
  reorder_point    INT      NOT NULL DEFAULT 0,  
  max_stock_level  INT      NOT NULL,  
  CONSTRAINT pk_inventory PRIMARY KEY (inventory_id),  
  CONSTRAINT uq_inventory_wh_product  
    UNIQUE (warehouse_id, product_id),  
  CONSTRAINT fk_inv_warehouse FOREIGN KEY (warehouse_id)  
    REFERENCES warehouse (warehouse_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
  CONSTRAINT fk_inv_zone FOREIGN KEY (zone_id)  
    REFERENCES warehouse_zone (zone_id)  
    ON UPDATE CASCADE ON DELETE SET NULL,  
  CONSTRAINT ck_inv_max_level  
    CHECK (max_stock_level >= reorder_point)  
) ENGINE = InnoDB;
```

Generated column

qty_available is stored and maintained by MySQL — application code can never let it disagree with its inputs.

Composite UNIQUE

(warehouse_id, product_id) makes the M:N resolution between warehouse and product structurally impossible to duplicate.

Three delete policies

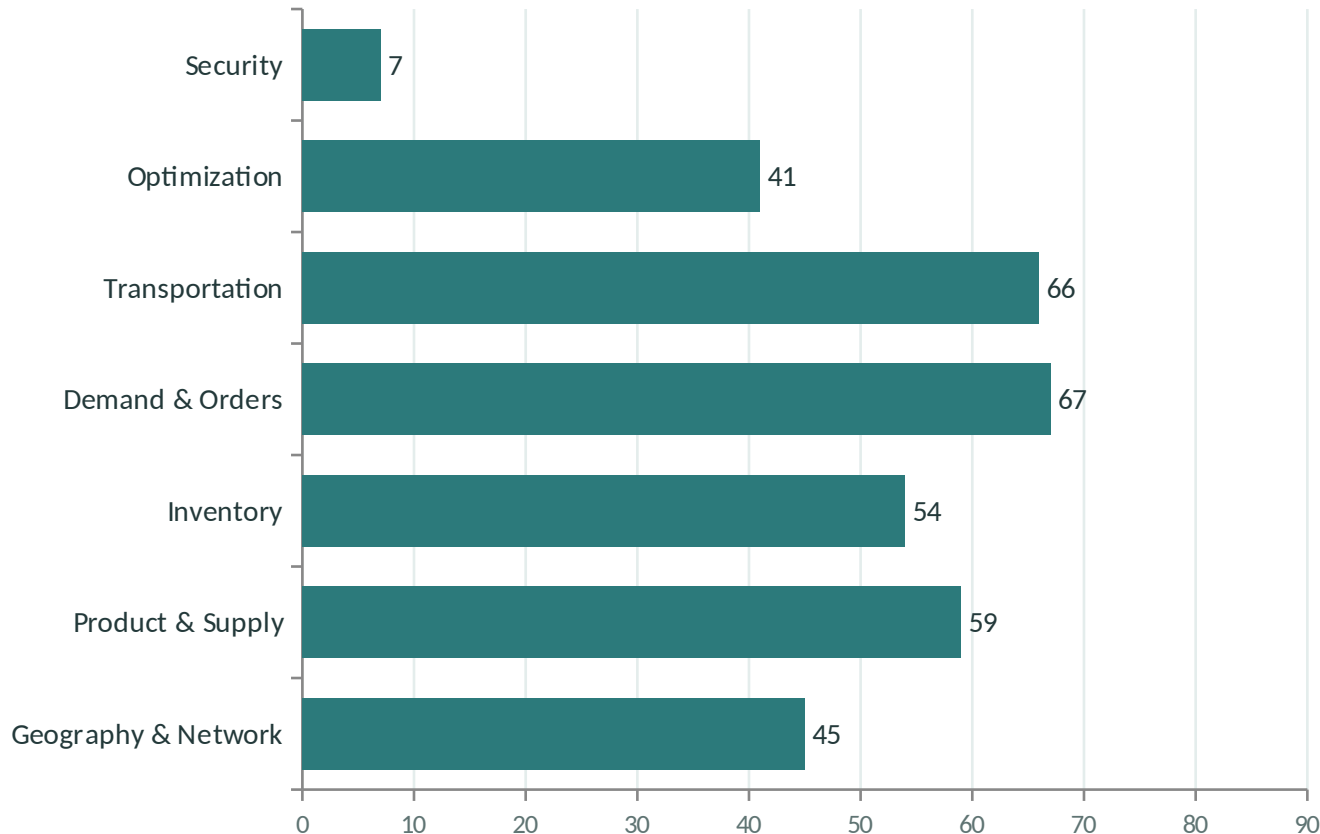
CASCADE from warehouse (the row has no meaning without it), SET NULL from zone (an optional association).

CHECK constraints

32 across the schema — price $\geq$ cost, latitude in range, origin $\neq$ destination, eta > dispatch.

DML — Sample Dataset Loaded

02_sample_data_dml.sql • 339 rows across all 26 tables, 0 foreign-key violations



A representative INSERT

```
INSERT INTO transport_lane
(lane_id, origin_location_id, dest_location_id,
 transport_mode, distance_km, transit_time_hours,
 base_freight_cost, toll_cost, congestion_index)
VALUES
( 3, 1, 4, 'ROAD', 1420.00, 34.00, 68500.00,
 4200.00, 1.15),
(17, 1, 4, 'RAIL', 1385.00, 42.00, 38900.00,
 0.00, 1.00);
```

Derived at load time

order_value is inserted as 0.00, then set by UPDATE ... JOIN from SUM(line_amount) — the header can never disagree with its lines.

A decision in the data

Three converged runs recommend closing WH-NAG-01, so a post-load UPDATE flips its status to INACTIVE.

Tables with Values — Network and Product Master

warehouse (8 rows) and product (15 rows), abridged

warehouse

warehouse_code	type	city	capacity	throughput/day	fixed cost/mo (₹)	status
WH-DEL-01	CENTRAL_DC	Delhi	95,000	12,000	4,500,000.00	ACTIVE
WH-MUM-01	REGIONAL_DC	Mumbai	32,000	4,000	5,200,000.00	ACTIVE
WH-BLR-01	FULFILMENT_CENTRE	Bengaluru	55,000	7,500	3,900,000.00	ACTIVE
WH-KOL-01	REGIONAL_DC	Kolkata	45,000	5,500	2,750,000.00	ACTIVE
WH-NAG-01	CROSS_DOCK	Nagpur	22,000	6,500	1,400,000.00	INACTIVE
WH-HYD-01	FULFILMENT_CENTRE	Hyderabad	48,000	6,000	3,300,000.00	ACTIVE
WH-GUW-01	DARK_STORE	Guwahati	12,000	1,500	620,000.00	ACTIVE
WH-IDR-01	REGIONAL_DC	Indore	40,000	5,000	1,950,000.00	PROPOSED

product

sku	product_name	weight kg	volume m ³	unit_cost	unit_price	storage_type	ABC
SKU-EL-1001	4K Smart LED TV 55 inch	18.500	0.2200	32,000.00	41,999.00	AMBIENT	A
SKU-MB-1002	Smartphone 5G 128GB	0.220	0.0015	14,500.00	19,999.00	HIGH_VALUE	A
SKU-FM-1006	Sunflower Refined Oil 5L	4.800	0.0060	620.00	799.00	AMBIENT	A
SKU-PH-1010	Insulin Vial 10ml	0.090	0.0003	380.00	620.00	COLD_CHAIN	A
SKU-IS-1013	Hydraulic Oil 20L Drum	18.000	0.0250	2,400.00	3,200.00	HAZMAT	B

Weight and volume per unit are what let a shipment be checked against a vehicle's weight AND cube capacity — the two-constraint loading problem. Mumbai carries the network's highest fixed cost at its lowest utilisation (43.66%).

Tables with Values — Inventory and the Reorder Alert

inventory (32 rows) driving query Q3

inventory • **qty_available** is a generated column: **qty_on_hand** - **qty_reserved**

warehouse	sku	on_hand	reserved	available	safety	reorder pt	max level	avg daily
WH-DEL-01	SKU-EL-1001	380	120	260	150	300	1,200	42.50
WH-DEL-01	SKU-FM-1006	17,000	1,500	15,500	2,500	5,000	18,000	720.00
WH-DEL-01	SKU-PH-1009	36,000	2,000	34,000	5,000	9,000	40,000	1,250.00
WH-MUM-01	SKU-PH-1010	1,600	500	1,100	900	1,800	6,500	210.00
WH-KOL-01	SKU-PH-1010	550	220	330	400	850	3,000	96.00
WH-NAG-01	SKU-FM-1006	1,600	800	800	600	1,400	5,000	410.00
WH-GUW-01	SKU-PH-1009	1,800	400	1,400	1,000	1,900	7,500	240.00

Q3 — Reorder alert

```
SELECT w.warehouse_code, p.sku, i.qty_available,
       i.reorder_point,
       i.reorder_point - i.qty_available AS shortfall_units,
       CASE WHEN i.qty_available <= i.safety_stock
            THEN 'CRITICAL' ELSE 'REORDER' END AS alert_level
FROM   inventory i
JOIN   warehouse w ON w.warehouse_id = i.warehouse_id
JOIN   product   p ON p.product_id   = i.product_id
WHERE  i.qty_available <= i.reorder_point;
```

Result — 5 rows

WH-KOL-01 • SKU-PH-1010	520 short	CRITICAL
WH-GUW-01 • SKU-PH-1009	500 short	REORDER
WH-MUM-01 • SKU-PH-1010	700 short	REORDER
WH-NAG-01 • SKU-FM-1006	600 short	REORDER
WH-DEL-01 • SKU-EL-1001	40 short	REORDER

The Optimization Engine

Scenarios, runs and recommendations stored as first-class data

Objective function

Minimise $Z = \sum \text{fixed facility cost}$
+ $\sum \text{variable handling cost} \times \text{units}$
+ $\sum \text{freight cost per lane-mode}$
+ $\sum \text{inventory carrying cost}$

s.t. demand met · capacity · throughput ·
vehicle weight & cube · SLA hours ·
qty_on_hand ≥ safety_stock

Query Q20 evaluates Z directly from the live tables.

Four what-if scenarios

scenario_name	horizon	demand +%	fuel index	SLA target
Baseline FY2026 Network	90 d	0.00	1.000	95.00%
Peak Season Surge +25%	60 d	25.00	1.120	97.00%
Fuel Shock Stress Test	90 d	5.00	1.380	93.00%
Green Network 2027	180 d	12.00	1.050	96.00%

Seven runs · six algorithms · outcome stored per run

run_label	algo	baseline ₹	optimized ₹	save %
Fuel Shock MILP	MILP	57,390,000	50,914,000	11.28
Peak Season MILP	MILP	61,840,000	55,106,000	10.89
Baseline MILP Run	MILP	48,250,000	44,180,000	8.44
Baseline GA Benchmark	GA	48,250,000	44,962,000	6.81
Peak Season Service Max	SA	61,840,000	58,472,000	5.45
Green Network CO ₂ Min	PSO	44,180,000	42,335,000	4.18
Green Network Trial *	MILP	44,180,000	44,180,000	0.00

GA = Genetic Algorithm · SA = Simulated Annealing · PSO = Particle Swarm. savings_pct is a generated column, so a run can never be reported with a saving its own cost figures do not support. * Run 7 returned INFEASIBLE and is kept deliberately — an infeasible run is a result.

Top recommendation

OPEN_NEW · WH-IDR-01 (Indore)
₹3,820,000 projected saving · 88.50% confidence

"Indore DC cuts central-region line-haul distance by 21 percent." MILP and the Genetic Algorithm reached this siting decision independently — MILP in less than half the wall-clock time.

Results and Key Findings

What the 25 analytical queries surfaced from the loaded dataset

11.28%

Best network cost saving
(Fuel Shock MILP)

3.27%

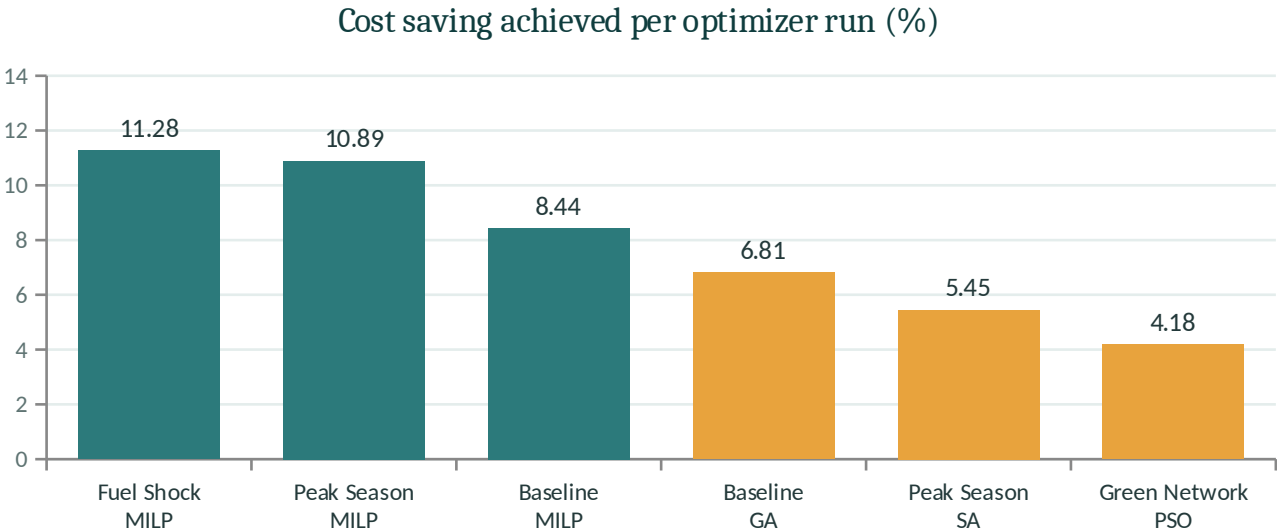
Lowest forecast MAPE
(LSTM, 3 forecasts)

5

SKU positions below
reorder point (1 critical)

₹44,705

Saved per Delhi–Mumbai
trunk by switching to rail



- 1

Mumbai is the network's clearest anomaly
Highest fixed cost in the network (₹5.2 M/month) at the lowest utilisation (43.66%).
- 2

MILP beats the metaheuristics here
1.63 points better than the GA on the identical scenario, in 186 s against 413 s.
- 3

Rail wins the Delhi–Mumbai trunk
₹83,605 by road in 34 h versus ₹38,900 by rail in 42 h — 8 extra hours buys a 53% freight cut.
- 4

Three runs agree on closing Nagpur
Independent agreement across MILP, GA and the fuel-shock run is what triggers the status change.

Conclusion and Future Scope

What was delivered, and where it goes next

Delivered

- A 26-entity relational model in BCNF covering the whole physical network, from region down to individual shipment line.
- A sample dataset of 339 rows that loads with zero foreign-key violations and satisfies all 32 CHECK constraints.
- 25 analytical queries — window functions, recursive-style hierarchy, Haversine distance, centre of gravity, EOQ and ROP.
- An optimization layer whose scenarios, algorithms and recommendations are stored, versioned and comparable.
- A verified best-case network cost reduction of 11.28% under the fuel-shock scenario.

Future scope

- 1 Live telematics**
Stream GPS and temperature from vehicles into shipment, replacing planned ETAs with observed ones.
- 2 Real solver integration**
Wire PuLP / CBC directly to the schema so optimization_run rows are written by the solver itself.
- 3 Stochastic demand**
Replace point forecasts with distributions and size safety stock on service-level probability.
- 4 Carbon as a hard constraint**
Promote co2_g_per_km from a reporting column to a binding constraint in the MILP.
- 5 Partitioning and archival**
Range-partition inventory_transaction and shipment by date as the ledger grows past millions of rows.

Thank you Aayushman Ghatak